```
$ git log --oneline && git revert HEAD
```

# Revert vs Reset

// 時光機的兩種開法：溫柔倒帶，還是橡皮擦擦掉

| **revert** | **reset** | **reflog** | **push -f** |
| --- | --- | --- | --- |
| 溫柔模式 | 狠心模式 | 救命符 | 危險動作 |

$ cat agenda.md

# 今日路線

**01 / 為什麼要回退**
血淚史盤點

**02 / Revert**
反向提交，歷史保留

**03 / Reset**
移動 HEAD，三種模式

**04 / 決策與救援**
決策樹、reflog、merge commit

目標：回退前先問一句「推送了沒？」，答案就出來一半。

PART 01

# 為什麼需要回退？

都是血淚史

# 常見情境

**推錯了**
錯誤的功能已推送到遠端（糟糕！）

**寫壞了**
某個提交導致 bug（誰寫的！...是我）

**先拿掉**
暫時移除某些改動以便測試

WRONG

直接刪除程式碼再提交 — 無法追溯，跳進火坑

RIGHT

用 Git 工具安全回退 — 優雅地後退

PART 02

# Revert：生成反向提交

倒帶鏡頭，但原片還在

# 運作原理




**新的提交**
內容是「撤銷 B 的改動」

**歷史完整**
A → B → C → Revert B，B 仍存在

像拍電影的倒帶鏡頭 — 觀眾看得到你倒帶，這就是可追溯性。

# 基本用法

```
# 回退單個提交
git revert <commit-id>

# 回退最近一次提交
git revert HEAD

# 回退但不自動提交
git revert --no-commit <commit-id>
```

# 批量回退

```
# A → B → C → D → E
# 想回退 C、D、E

git revert --no-commit b2c3d4e..e5f6g7h
git commit -m "Revert changes C to E"
git push origin main
```

**--no-commit**
多個 revert 合併成一個提交；範圍 <old>..<new> 不含 old

PART 03

# Reset：重寫歷史

橡皮擦，擦掉就沒了

# 運作原理




**移動 HEAD 指針**
改變提交歷史本身

**歷史變成 A → B**
C 被完全移除，消失了

# 三種模式：擦到哪一層？

| 模式 | 影響範圍 | 使用情境 |
| --- | --- | --- |
| `--soft` | 只移動 HEAD | 改動保留在**暫存區**，可直接重新提交 |
| `--mixed`（預設） | HEAD + 暫存區 | 改動保留在**工作區**，需重新 add |
| `--hard` | HEAD + 暫存區 + 工作區 | **完全刪除改動**，謹慎使用！ |

DANGER — 核彈按鈕

`--hard` 會永久刪除未提交的改動。按下去之前：

1. 備份了嗎？ 2. 真的不要了？ 3. 這分支只有我在用吧？

PART 04

# 對決與救援

選哪個？擦錯了怎麼辦？

# 正面對決

| 特性 | Revert | Reset |
| --- | --- | --- |
| 歷史記錄 | 保留（新增反向提交） | 重寫（刪除提交） |
| 安全性 | 高（溫柔模式） | 低（狠心模式） |
| 已推送的提交 | ✅ 推薦 | ❌ 避免 |
| 團隊協作 | 安全 | 危險 |
| 追蹤性 | 可追溯 | 無法追溯 |

一句話：公開的用 `revert`，私有的才 `reset`。

# 決策樹

CHECKPOINT_01

錯誤的提交「已推送到 main」，團隊五個人都拉了。怎麼回退？

A `git reset --hard HEAD~1`，然後 `push -f`

B `git revert <commit-id>`，然後 `push`

C 直接刪掉錯誤的程式碼，再提交一次

# 救援：reflog

```
# 誤用 reset --hard 刪了改動？
# 短時間內可以救回

git reflog
# 找到刪除前的 commit-id

git reset --hard <commit-id>
# 穿越回去
```

**reflog 是救命符**
記錄 HEAD 的每次移動，垃圾回收前都找得回來

# 兩個進階題

```
# Revert 一個 merge commit
git revert -m 1 <merge-commit-id>
# -m 1 = 保留主線（通常是 main）
```

```
# 隊友不小心 reset 了公開分支？
git fetch origin
git reset --hard origin/main
# 全員同步回遠端狀態
```

RULE

reset 後已推送的分支需要 `push -f` — 會影響他人，小心被追殺

# 今日重點

**Revert**
反向提交、保留歷史，已推送的首選

**Reset**
重寫歷史，只用在本地未推送

**批量回退**
revert --no-commit <old>..<new>

**reflog**
--hard 擦錯了，短時間內救得回

要反悔時，先找溫柔的 `revert`，真的沒辦法再狠心 `reset`。

# History preserved.

下一步：git branch 與 commit 訊息規範

```
$ git log --oneline --graph
revert → push | reset → reflog → 救回
```